CSCI 2406: Topic 07

# Branching, Iteration, and Control Flow

Comparisons, condition flags, conditional branches, loops, and translation of high-level control structures

Computer Organization & Architecture | AArch64 Reference

# Topic 7 Outline

**Topic Objective**

Explain how AArch64 changes program execution order using comparisons, condition flags, branches, and loop patterns.

- **Section 1**: Sequential execution and branch targets
- **Section 2**: Comparisons and condition flags
- **Section 3**: Conditional and unconditional branches
- **Section 4**: Translating `if`, `else`, and loops
- **Section 5**: Specialized branch forms
- **Section 6**: Summary and self-test

Section 01

# Sequential Execution and Branch Targets

# The Program Counter Controls Instruction Order

## Program Counter

The program counter identifies the instruction address associated with the current control-flow position. In AArch64, it is not an ordinary general-purpose register.

- Sequential execution normally proceeds to the next 32-bit A64 instruction.
- A branch replaces that ordinary next-instruction behavior with a target address.
- Calls and returns are controlled changes in the same instruction stream.
- Exceptions can also redirect control flow, but that belongs to later systems material.

## A64 Instruction Width

Because A64 instructions are 4 bytes wide, sequential execution commonly advances by 4 bytes before branch redirection is considered.

# Labels Name Instruction Addresses

## Assembler Labels

A label is a symbolic name for an instruction address. The assembler resolves the label into the branch target encoded in the machine instruction.

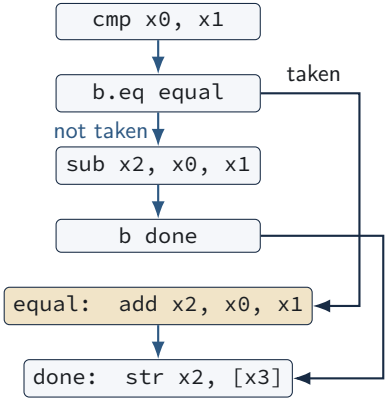
## Assembly Pattern

```
loop:
subs x0, x0, #1
b.ne loop
```

## Meaning

- `loop:` marks an address.
- `b.ne loop` can redirect execution back to that address.
- The label does not occupy a machine instruction by itself.

# Control Flow Changes the Instruction Stream




## Branch Outcome

A conditional branch either transfers control to its target or falls through to the next sequential instruction.

Section 02

# Comparisons and Condition Flags

# cmp Updates Flags by Subtraction

## Underlying Operation

`cmp x0, x1` performs a subtraction for flags and discards the numeric result.

```
cmp x0, x1
subs xzr, x0, x1
```

- The computed value `x0 - x1` is not kept as a data result.
- NZCV is updated from the subtraction.
- Later conditional branches inspect those flags.
- This connects comparison directly to the flag-setting arithmetic from Topic 5.

# The NZCV Flags Used by Branches


- **Z**: equality after a compare.
- **N** and **V**: signed ordering after subtraction.
- **C**: unsigned ordering after subtraction; `C=1` means no unsigned borrow.
- The same compare can support signed or unsigned branch decisions.

# Equality Branches

## Equal

`b.eq target`

- Branches when `Z=1`.
- Used after `cmp`, `subs`, or another flag-setting operation.

## Not Equal

`b.ne target`

- Branches when `Z=0`.
- Common for loop continuation and mismatch tests.

## Example

```
cmp x0, x1
b.eq equal_case
```

# Signed and Unsigned Ordering

| Branch | Interpretation | Condition after `cmp x0, x1` |
|---|---|---|
| `b.lt label` | signed less than | `x0 < x1` as two's complement, $N \neq V$ |
| `b.ge label` | signed greater/equal | `x0 >= x1` as two's complement, $N = V$ |
| `b.lo label` | unsigned lower | `x0 < x1` as unsigned, $C = 0$ |
| `b.hs label` | unsigned higher/same | `x0 >= x1` as unsigned, $C = 1$ |

**Important Distinction**

Do not use signed and unsigned branch mnemonics interchangeably. They inspect different flag evidence.

# Same `cmp`, Different Meaning

## Example Bit Patterns

For 8-bit intuition, compare `0xff` with `0x01`. The bit patterns are the same, but the numerical interpretation changes.

| Interpretation | Values | Correct ordering |
|---|---|---|
| Unsigned | $255$ and $1$ | $255 > 1$ |
| Signed two's complement | $-1$ and $1$ | $-1 < 1$ |

## AArch64 Consequence

After the same comparison, `b.lo` and `b.lt` may lead to different control-flow paths.

Section 03

# Conditional and Unconditional Branches

# Unconditional Branch

## Branch Always

`b label` transfers control to the target label without testing NZCV flags.

```
b done
add x0, x0, #1
done:
```

- Used to skip over an alternative block.
- Used at the bottom of loops to return to the loop test or loop body.
- Does not depend on signed or unsigned interpretation.

# Conditional Branch Pattern

## Two-Step Decision

A common AArch64 decision first sets flags, then uses a branch instruction to test those flags.

## Step 1: Compare

```
cmp x0, x1
```

- Computes `x0 - x1` for flags.
- Updates NZCV.
- Does not keep the subtraction result.

## Step 2: Branch

```
b.lt less
```

- Tests the relevant flag condition.
- Redirects control if the condition is true.
- Falls through otherwise.

# Common Conditional Branch Mnemonics

| Branch | Meaning | Branch | Meaning |
|---|---|---|---|
| `b.eq` | equal | `b.ne` | not equal |
| `b.lt` | signed less than | `b.ge` | signed greater/equal |
| `b.gt` | signed greater than | `b.le` | signed less/equal |
| `b.lo` | unsigned lower | `b.hs` | unsigned higher/same |
| `b.hi` | unsigned higher | `b.ls` | unsigned lower/same |

## Mnemonic Choice

The comparison operation may be identical, but the branch mnemonic determines how the flags are interpreted.

# Calls and Returns

## Branch with Link

```
bl function
```

- Branches to a function label.
- Saves the return address in `x30`.
- `x30` is also called `lr`.

## Return

```
ret
```

- Returns using the link register by default.
- Control resumes after the original `bl`.
- Stack discipline is covered in procedure topics.

## Function-Call State

A call changes control flow and also creates a calling-convention obligation to preserve the right registers.

Section 04

# Translating Selection and Iteration

# Translating an `if` Statement

**C**

```
if (x == y) {
  z = x + y;
}
```

**AArch64**

```
cmp x0, x1
b.ne endif
add x2, x0, x1
endif:
```

**Control-Flow Shape**

The branch often tests the opposite of the high-level condition so execution can skip the body when the condition is false.

# Translating `if-else`

## C

```
if (x == y) {
  z = x + y;
} else {
  z = x - y;
}
```

## AArch64

```
cmp x0, x1
b.ne else_case
add x2, x0, x1
b end_if
else_case:
sub x2, x0, x1
end_if:
```

## Unconditional Skip

The unconditional branch after the true block prevents fall-through into the false block.

# Translating a `while` Loop

## C

```
while (i < n) {
  sum += a[i];
  i++;
}
```

## AArch64 Skeleton

```
loop_test:
cmp x1, x2
b.ge loop_done
// loop body
add x1, x1, #1
b loop_test
loop_done:
```

## Test Before Body

A `while` loop checks its condition before executing the body, so the body may execute zero times.

# Translating a `do-while` Loop

## C

```
do {
  sum += *p;
  p++;
} while (*p != 0);
```

## AArch64 Skeleton

```
loop_body:
// loop body
ldr x3, [x0]
cmp x3, xzr
b.ne loop_body
```

## Test After Body

A `do-while` loop executes the body once before the first condition test.

# Counted Loop with subs

## Decrement and Test

A decrementing loop can combine counter update and flag update using subs.

```
loop:
// loop body
subs x0, x0, #1
b.ne loop
```

- subs decrements x0 and updates NZCV.
- b.ne repeats while the result is not zero.
- This works naturally when the initial count is positive.

# Array Traversal Loop

## Example

Sum n 64-bit integers. Let `x0` hold the base pointer, `x1` hold n, and `x2` hold the running sum.

```
mov x2, xzr
loop:
cbz x1, done
ldr x3, [x0], #8
add x2, x2, x3
sub x1, x1, #1
b loop
done:
```

## Pointer Update

`ldr x3, [x0], #8` loads the current element and advances the pointer by one 64-bit element.

# String Traversal Loop

## Null-Terminated Strings

A C string ends with a zero byte. Byte loads are used so the loop tests one character at a time.

```
loop:
ldrb w1, [x0], #1
cbz w1, done
// process character in w1
b loop
done:
```

- `ldrb` reads one byte and zero-extends into `w1`.
- Post-indexing advances `x0` after the load.
- `cbz` tests for the null terminator without modifying NZCV.

Section 05

# Specialized Branch Forms

# Compare and Branch on Zero

## Branch if Zero

```
cbz x0, target
```

- Tests whether `x0` equals zero.
- Branches if the test is true.
- Does not update NZCV flags.

## Branch if Nonzero

```
cbnz x0, target
```

- Tests whether `x0` is nonzero.
- Branches if the test is true.
- Useful for pointer and counter tests.

## Technical Precision

These are architectural instructions that combine a zero test with a branch. Do not describe their speed as universally one cycle; latency depends on the processor implementation.

# Test Bit and Branch

## Bit-Specific Control Flow

`tbz` and `tbnz` test a selected bit in a register and branch based on whether that bit is zero or nonzero.

## Test Bit Zero

```
tbz x0, #3, clear
```

- Tests bit 3 of `x0`.
- Branches if the bit is 0.
- Does not update NZCV.

## Test Bit Nonzero

```
tbnz x0, #3, set
```

- Tests bit 3 of `x0`.
- Branches if the bit is 1.
- Useful for flags packed in a word.

# Branches and Flags

| Branch form | Tests | NZCV effect |
|---|---|---|
| `b.cond label` | Existing condition flags | Reads NZCV; does not update it |
| `cbz/cbnz` | Register equals zero or nonzero | Does not update NZCV |
| `tbz/tbnz` | One selected register bit | Does not update NZCV |
| `b label` | No condition | Does not test or update NZCV |

**Programming Implication**

Use `b.cond` when flags already contain the comparison needed. Use `cbz`, `cbnz`, `tbz`, or `tbnz` when a direct register test is clearer.

# Control Flow and Performance

## Branch Cost Is Implementation-Dependent

AArch64 specifies branch behavior, not a universal branch timing. Performance depends on the microarchitecture and workload.

- Sequential instruction fetch is simple when the next address is predictable.
- Conditional branches introduce uncertainty into the instruction stream.
- Branch prediction tries to keep the pipeline supplied with useful instructions.
- A mispredicted branch may discard speculative work and increase effective CPI.

## Course Connection

Branches connect architectural control flow to later microarchitecture topics such as pipelines, prediction, and hazards.

# Common Branching Mistakes

- Using `b.lt` for an unsigned comparison that should use `b.lo`.
- Forgetting that `cmp x0, x1` means the flags reflect `x0 - x1`, not `x1 - x0`.
- Assuming `cbz`, `cbnz`, `tbz`, or `tbnz` update NZCV flags.
- Omitting the unconditional branch needed to skip an `else` block.
- Placing a loop update after a branch target where it no longer executes on every iteration.

**Debugging Practice**

Trace the values, then trace the flags, then trace the branch target. Avoid guessing from the high-level source alone.

Section 06

# Summary and Self-Test

# Topic 7 Summary

- The program counter determines the current control-flow position, but it is not an ordinary AArch64 general-purpose register.
- `cmp` updates NZCV by performing a flag-setting subtraction and discarding the result.
- Conditional branches use flags, while `cbz`, `cbnz`, `tbz`, and `tbnz` test registers directly without updating NZCV.
- Signed and unsigned branch mnemonics must be selected according to the intended interpretation.
- High-level `if`, `else`, `while`, and `do-while` constructs translate into labels, comparisons, conditional branches, and fall-through paths.

# Self-Test 1/2

1. What does `cmp x0, x1` compute internally, and where is the result stored?
2. Why can the same `cmp` support both signed and unsigned comparisons?
3. Which branch should be used for unsigned lower-than: `b.lt` or `b.lo`?
4. What does `b.eq` test after a comparison?
5. Why is an unconditional `b` often needed in an `if-else` translation?

# Self-Test 2/2

6. How does `cbz x0, target` differ from `cmp x0, xzr` followed by `b.eq target`?
7. Why does `ldrb w1, [x0], #1` help with string traversal?
8. What is the difference between a `while` loop and a do-`while` loop in branch structure?
9. What role does `x30` play after a `bl` instruction?
10. Why should branch timing not be described as a fixed ISA property?

# Review Exercise

## Classify the Branch Behavior

For each instruction, state whether it reads NZCV, updates NZCV, directly tests a register, or branches unconditionally.

| Instruction | Classification |
|---|---|
| `b.eq target` | |
| `cbz x0, target` | |
| `tbz x1, #4, target` | |
| `b target` | |
| `cmp x0, x1` | |

# Next Topic

**Procedures and the Stack**

The next lecture applies control flow to function calls, register-saving conventions, stack frames, and procedure-level organization.

- Function calls using `bl`
- Return addresses and `x30`
- Stack pointer updates
- Caller-saved and callee-saved registers
- Stack frame layout